DATA STRUCTURES · DIGITAL NOTES

UNIT I

Introduction to Linear Data Structures

Linear data structures · ADTs · Complexity analysis · Searching · Sorting

Roadmap of this unit

1.1	Linear data structures: definition & importance
1.2	Abstract Data Types (ADTs) & implementation
1.3	Time and space complexity analysis
1.4	Searching: Linear search & Binary search
1.5	Sorting: Bubble, Selection & Insertion sort

1.1 Linear Data Structures

DEFINITION

Data structure = a way of **organising, storing and managing data** in memory so that it can be used **efficiently**.

Linear data structure = a data structure in which elements are arranged **one after another in a sequence**. Every element has exactly one **predecessor** (except the first) and one **successor** (except the last).

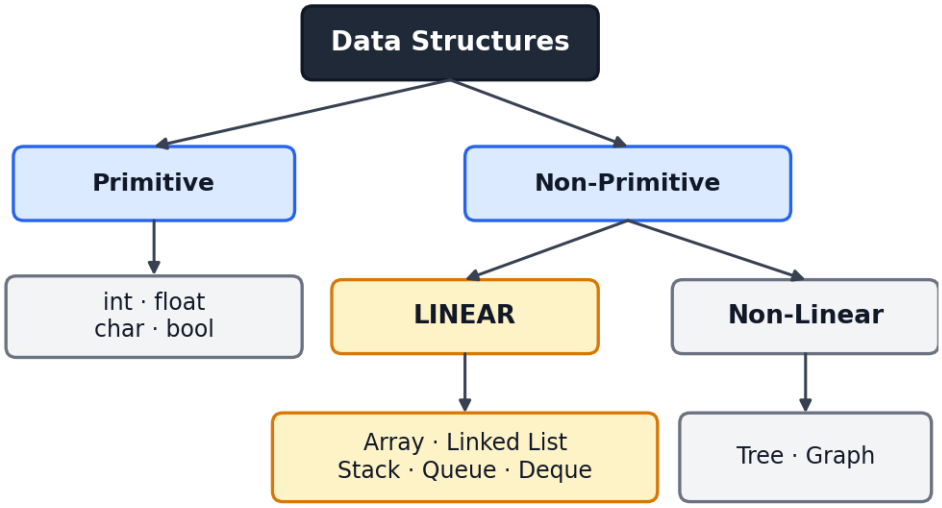


Fig: Classification of data structures – the highlighted branch is the focus of this subject

Why are linear data structures important?

- **Foundation** – every advanced structure (trees, graphs, hash tables) is built using arrays or linked lists.
- **Simple & natural** – real-life data is sequential: a queue at a ticket counter, a stack of plates, a playlist.
- **Easy traversal** – visit all elements in one pass, from first to last.
- **Efficient memory use** – choose static (array) or dynamic (linked) allocation as the problem needs.
- **Used everywhere** – OS scheduling, undo/redo, browser history, music players, expression evaluation.

Characteristics (Properties)

- Elements are stored in a **linear order** (single level).
- Only **one element** can be reached directly after another – traversal is **sequential**.
- Can be implemented in **two ways**: **contiguous memory** (arrays) or **linked memory** (pointers).
- Operations: **insert, delete, traverse, search, sort**.

Array – the basic linear structure

An **array** is a collection of **same-type** elements stored in **contiguous memory locations**, accessed using an **index**.

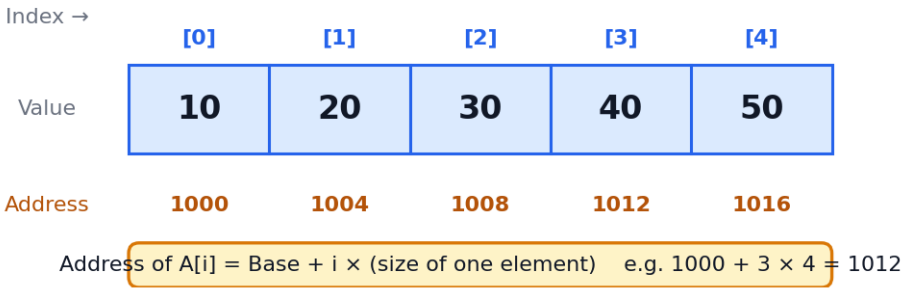


Fig: Array stored in contiguous memory – address calculation for any index is instant

Feature	Linear	Non-linear
Arrangement	Sequential, one level	Hierarchical / network, many levels
Traversal	Single run (one pass)	Needs multiple paths / recursion
Relationship	One-to-one	One-to-many or many-to-many
Memory use	Simple, predictable	More complex
Examples	Array, List, Stack, Queue	Tree, Graph

Basic operations on a linear structure

Operation	Meaning	Array cost
Traversal	Visit every element once	O(n)
Access	Read / write element at index i	O(1)
Insertion	Add an element at a position (shift right)	O(n)
Deletion	Remove an element (shift left)	O(n)
Searching	Find the location of a value	O(n) / O(log n)
Sorting	Arrange in order	O(n log n) – O(n^2)

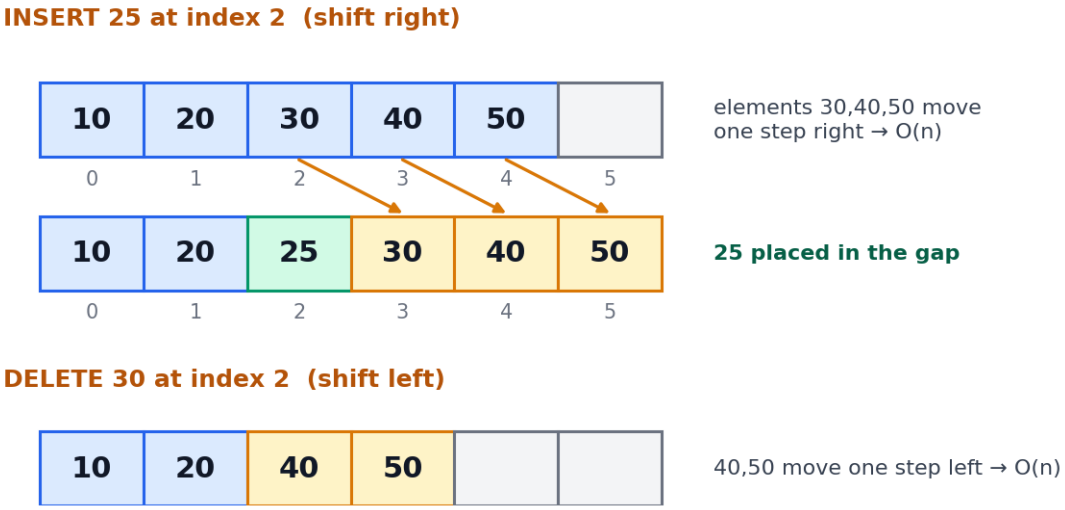


Fig: Insertion and deletion in an array need shifting of elements

REMEMBER

- Array size is **fixed** at creation (static) – cannot grow later.
- Insert/delete in the **middle is costly** because of shifting; access by index is **very fast**.

1.2 Abstract Data Types (ADT)

DEFINITION

ADT = a **logical description** of *what* a data type stores and *what operations* can be done on it – without saying *how* they are implemented.

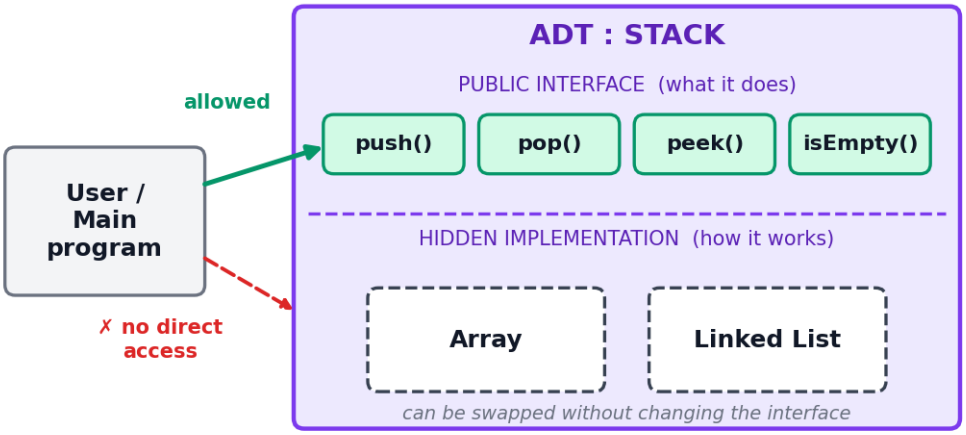


Fig: ADT block diagram – the user sees only the interface; the implementation stays hidden

Key points

- An ADT = **Data** (values it holds) + **Operations** (what you can do) + **Rules** (behaviour, e.g. LIFO).
- **Abstraction** – show the essential features, hide the details.
- **Encapsulation** – data and its operations are bundled; data is accessed only through the interface.
- Same ADT can have **many implementations** (Stack → array or linked list).

ADT	Data stored	Main operations	Rule
List	Sequence of items	insert, delete, search, traverse	Order maintained
Stack	Items in a pile	push, pop, peek, isEmpty	LIFO
Queue	Items in a line	enqueue, dequeue, front	FIFO

ADT vs Data Structure

ADT	Data Structure
Logical view – *what*	Physical view – *how*
Defined by operations	Defined by memory layout & code
Language independent	Written in a language (C, Java…)
Example: Stack ADT	Example: stack using array

Implementing an ADT (example: Stack in C)

```
#define MAX 100
typedef struct {            // DATA part
    int items[MAX];
    int top;
} Stack;

void init(Stack *s)           { s->top = -1; }                // OPERATIONS part
int  isEmpty(Stack *s)        { return s->top == -1; }
void push(Stack *s, int x)    { s->items[++(s->top)] = x; }
int  pop(Stack *s)            { return s->items[(s->top)--]; }
```

The user calls push() / pop() only – never touches items[] or top directly

Advantages of using ADTs

- **Reusability** – write once, use in many programs.
- **Easy maintenance** – change the implementation without touching user code.
- **Security** – internal data cannot be corrupted from outside.
- **Clarity** – programmers think in terms of operations, not memory.

REMEMBER

Steps to implement an ADT: ① define the data → ② define the operations → ③ choose the structure (array / linked list) → ④ write the functions → ⑤ test.

1.3 Time and Space Complexity

DEFINITIONS

- **Algorithm** = a finite, step-by-step procedure to solve a problem.
- **Time complexity** = how the **running time grows** as the input size **n** grows (count of basic steps, *not* seconds).
- **Space complexity** = how the **extra memory** needed grows with n (input space + auxiliary space).

Asymptotic notations

Notation	Meaning	Used for
Big-O O( )	**Upper bound** – never slower than this	Worst case (most common)
Omega Ω( )	**Lower bound** – never faster than this	Best case
Theta Θ( )	**Tight bound** – upper and lower both match	Average / exact growth

Best, average and worst case

- **Best case** – input that makes the algorithm fastest (e.g., key is the first element).
- **Average case** – typical input.
- **Worst case** – input that makes it slowest (e.g., key is absent). **We usually report this.**

How to count steps

- Simple statement → **1 step** → O(1)
- Single loop running n times → **n steps** → O(n)
- Loop inside a loop (both n) → **n × n** → O(n^2)
- Loop where the counter doubles/halves (i = i×2) → O(log n)
- Sequential blocks → **add** and keep the largest term. **Drop constants:** 5n + 3 → O(n)

```
for (i = 1; i <= n; i++)          // runs n times
    for (j = 1; j <= n; j++)      // runs n times for each i
        count++;                  // executes n x n times  =>  O(n^2)

for (i = 1; i < n; i = i * 2)     // i = 1,2,4,8...  =>  log2(n) times  =>  O(log n)
    count++;
```

Two examples of counting steps

Worked example – total time of a function

```
int sumArray(int a[], int n) {
    int sum = 0;                  // 1 step
    for (int i = 0; i < n; i++)   // loop runs n times
        sum += a[i];              // n steps
    return sum;                   // 1 step
}                                 // T(n) = n + 2   =>  O(n)     Space: sum, i  =>  O(1)
```

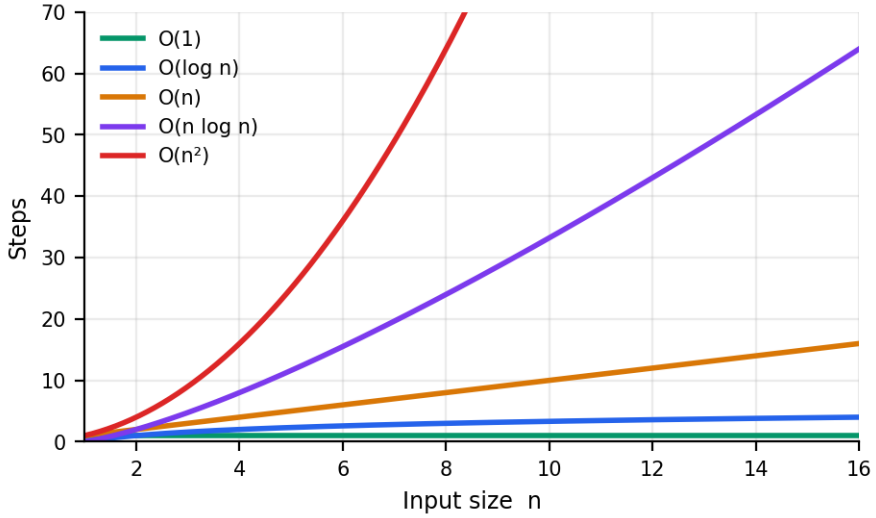


Fig: Growth of common complexities – smaller curve = faster algorithm

Order	Name	Example
O(1)	Constant	Array access a[i]
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, traversal
O(n log n)	Linearithmic	Merge sort
O(n^2)	Quadratic	Bubble, Selection, Insertion sort

Space complexity

- **In-place** algorithm uses only a few extra variables → **O(1) space** (bubble, selection, insertion sort).
- Algorithm that creates a copy of the input uses **O(n) space**.
- Recursion uses the call stack → recursion of depth n costs O(n) space.

EXAM TIP
Order to remember: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$. Always write complexity as a **function of n**.

1.4 Searching Techniques

DEFINITION
Searching = finding the location of a given element (**key**) in a collection. Result: **successful** (index returned) or **unsuccessful** (e.g., −1).

A. Linear (Sequential) Search

- Compare the key with each element **one by one**, from the first to the last.
- Works on **sorted or unsorted** data.
- Simple but slow for large data.

Algorithm

1. Start from index **i = 0**.
2. If **a[i] == key** → return i (found).
3. Else **i = i + 1**; repeat until i = n.
4. If the loop ends → return −1 (not found).



Fig: Linear search for key 35 – found in the 3rd comparison

```
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;     // found
    return -1;                         // not found
}
```

Case	Time	When
Best	O(1)	Key is at the first position
Average	O(n)	Key in the middle
Worst	O(n)	Key is last or absent
Space	O(1)	No extra memory

EXAM TIP

- Linear search is also the **only** choice for an **unsorted linked list**.
- If the key occurs many times, the basic version returns the **first** match.

B. Binary Search

COMMON MISTAKE

Binary search works **only on a SORTED array**. On unsorted data the answer will be wrong.

- Uses **divide and conquer**: compare the key with the **middle** element and discard half of the array each time.
- **mid = low + (high − low) / 2**
- If key = a[mid] → found. If key > a[mid] → **low = mid + 1**. If key < a[mid] → **high = mid − 1**.

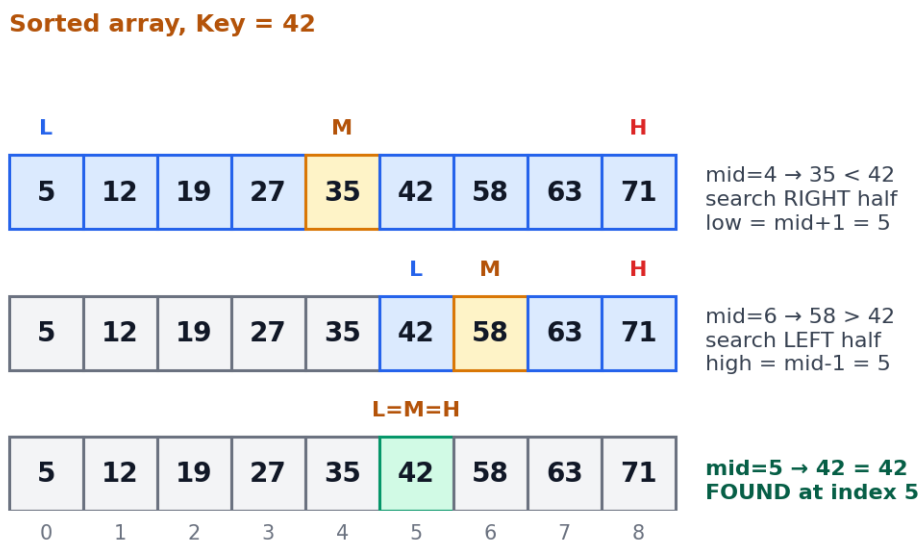


Fig: Binary search for key 42 – only 3 comparisons for 9 elements

```
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key)      return mid;      // found
        else if (a[mid] < key)  low  = mid + 1;  // go right
        else                    high = mid - 1;  // go left
    }
    return -1;                                   // not found
}
```

- Each step halves the data: n → n/2 → n/4 … → 1. Steps = **$\log_2 n$**. For 1,00,000 elements → only **≈ 17** comparisons.

Basis	Linear Search	Binary Search
Data must be sorted?	No	**Yes**
Technique	Sequential scan	Divide and conquer
Best / Worst time	O(1) / O(n)	O(1) / O(log n)
Works on	Array, linked list	Array (random access needed)
Best for	Small or unsorted data	Large sorted data

1.5 Sorting Techniques

DEFINITION
Sorting = arranging elements in **ascending** or **descending** order.

- **Stable sort** – equal elements keep their original relative order.
- **In-place sort** – needs only O(1) extra memory.
- **Pass** – one complete round of the outer loop.
- Sorted data makes searching (binary search) very fast.

A. Bubble Sort

- Repeatedly **compares adjacent elements** and **swaps** them if they are in the wrong order.
- After pass 1 the **largest** element "bubbles up" to the last position; after pass 2 the second largest, and so on.
- **Optimisation:** if a pass makes **no swap**, stop – the array is already sorted.



Fig: Bubble sort on [5, 1, 4, 2, 8] – green = fixed in final position

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {                // wrong order -> swap
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;                       // already sorted
    }
}
```

B. Selection Sort

- Find the **minimum** element in the unsorted part and **swap** it with the first unsorted position.
- The sorted part grows from the **left** by one element every pass.
- Only **n − 1 swaps** in total – good when swapping is costly.



Fig: Selection sort on [64, 25, 12, 22, 11] – amber = position i, red = minimum found

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;            // remember smallest index
        int t = a[i]; a[i] = a[min]; a[min] = t;   // one swap per pass
    }
}
```

C. Insertion Sort

- Works like **arranging playing cards** in your hand.
- Take the next element (**key**) and **insert** it at its correct place in the already-sorted left part by **shifting** bigger elements right.
- Very fast on **nearly sorted** data – best case O(n).



Fig: Insertion sort on [7, 4, 5, 2] – green = sorted part, amber = key being inserted

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {   // shift larger elements right
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                  // place the key
    }
}
```

Comparison of sorting techniques

Algorithm	Best	Average	Worst	Space	Stable?	Remark
Bubble	O(n)*	$O(n^2)$	$O(n^2)$	O(1)	Yes	Simple; many swaps
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Fewest swaps
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Best for nearly sorted data

* with the “no swap” optimisation.

Which sort should I use?

Situation	Best choice	Reason
Data is nearly sorted	Insertion sort	Very few shifts → close to O(n)
Writes / swaps are expensive	Selection sort	Only n − 1 swaps
Very small data or teaching	Bubble sort	Easiest to code and understand
Large data	Merge / Quick sort	O(n log n) – beyond this unit

Applications of searching and sorting

- **Searching:** finding a contact in a phone book, looking up a student by roll number, checking a login name.
- **Sorting:** ranking exam results, ordering products by price, arranging files by date, preparing data for binary search.

Try it yourself (dry-run practice)

1. Apply **bubble sort** to 9, 3, 7, 1 and write the array after each pass.
2. Apply **selection sort** to 29, 10, 14, 37, 13 and list every swap.
3. Apply **insertion sort** to 8, 5, 6, 2 – show the shifts for each key.
4. Search key **27** in [3, 9, 14, 21, 27, 33, 40] using binary search – write low, mid, high for every step.
5. How many comparisons does binary search need in the worst case for n = 1024? *(Answer: 11)*

1.6 Quick Revision & Practice

ONE-PAGE SUMMARY

- Linear DS → elements in a **sequence** (array, linked list, stack, queue).
- ADT → **what** (operations) not **how** (implementation).
- Time complexity → growth with n; use **Big-O**; drop constants.
- Linear search O(n) – any data. Binary search O(log n) – **sorted** data only.
- Bubble = swap neighbours · Selection = pick minimum · Insertion = place into sorted part.
- All three simple sorts: worst case **O(n²)**, space **O(1)**.

Likely exam questions

2 marks

- Define a linear data structure. Give two examples.
- What is an ADT?
- Define time complexity and space complexity.
- Why does binary search need sorted data?
- What is a stable sort?

5 marks

- Differentiate between linear and non-linear data structures.
- Explain Big-O, Omega and Theta notations.
- Write the algorithm for linear search and analyse its complexity.
- Trace insertion sort on 9, 5, 1, 4, 3.

10 marks

- Explain binary search with an example and derive its time complexity.
- Explain bubble sort, selection sort and insertion sort with programs and compare them.
- Explain ADT with the help of a Stack ADT example.